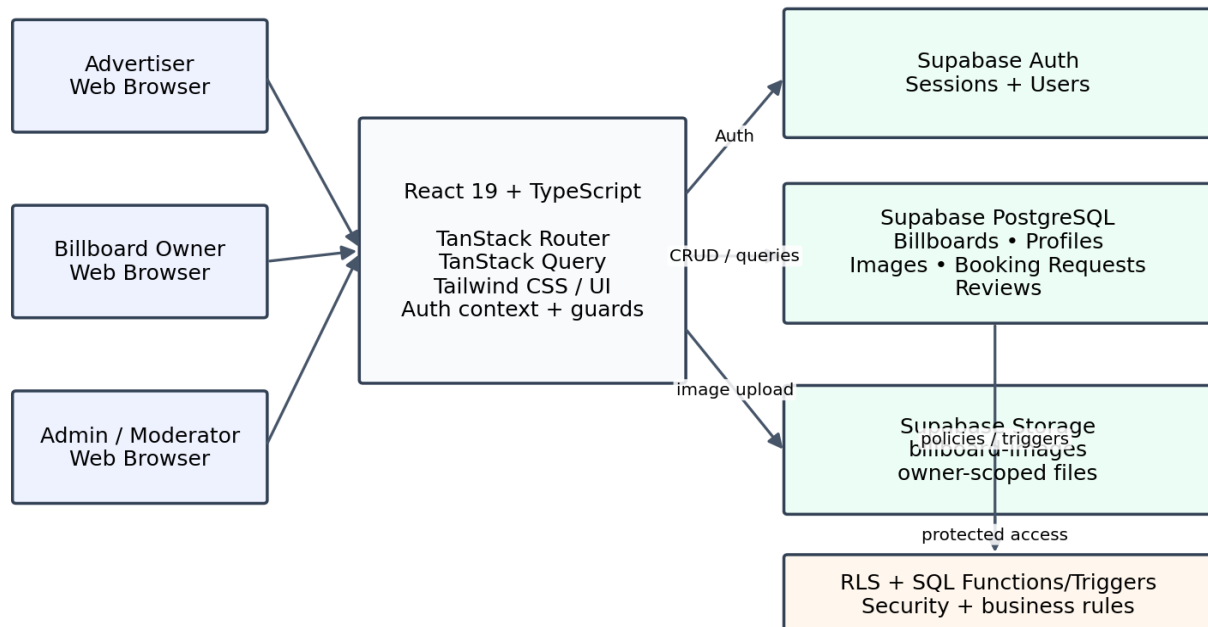


Panorama — Billboard Marketplace

Hackathon Project Documentation

Panorama — System Architecture



1. Abstract

Panorama is a web-based marketplace that connects billboard owners with advertisers. Owners can create, manage, publish and maintain billboard listings with location, dimensions, lighting, pricing, availability and images. Advertisers can browse published spaces, search and filter inventory, view details, submit date-based booking requests and track request status. The platform uses Supabase for authentication, PostgreSQL data, row-level security, storage and server-side business rules.

2. Problem Statement

Traditional outdoor advertising procurement is fragmented: advertisers may need to contact multiple billboard owners, compare locations and prices manually, and coordinate availability through separate channels. Billboard owners likewise lack a focused digital marketplace for publishing inventory and receiving structured booking requests.

3. Proposed Solution

Panorama provides a centralized marketplace where billboard inventory is represented as structured listings. Advertisers discover spaces and submit date-based requests, while owners manage listings and respond to requests. Database-level controls protect ownership and enforce important business rules instead of relying only on the browser.

4. Objectives

- Digitize billboard discovery and inventory presentation.
- Reduce friction in comparing location, dimensions, lighting, price and availability.
- Give billboard owners a self-service listing and request-management workflow.
- Provide authenticated booking-request tracking for advertisers.
- Protect user data and business rules using Supabase RLS and PostgreSQL functions/triggers.

5. Key Features

- Public browsing of published billboard listings without requiring an account.
- Search, city/type/price/dimension/lighting filters and sorting.
- Billboard detail pages with photos, location, pricing, dimensions, availability and owner information.
- Advertiser booking requests with start/end dates and optional campaign message.
- Owner dashboard with listing and booking-request statistics.
- Seven-step billboard creation/editing workflow with image upload, preview, ordering and availability dates.
- Listing lifecycle: Draft → Published / Unpublished, with owner-only management.
- Owner-scoped image storage in the Supabase billboard-images bucket.
- Booking management for owners and request tracking for advertisers.
- Verified reviews: only advertisers with ended accepted bookings can review; owners may reply once; moderators can hide reviews with a reason.

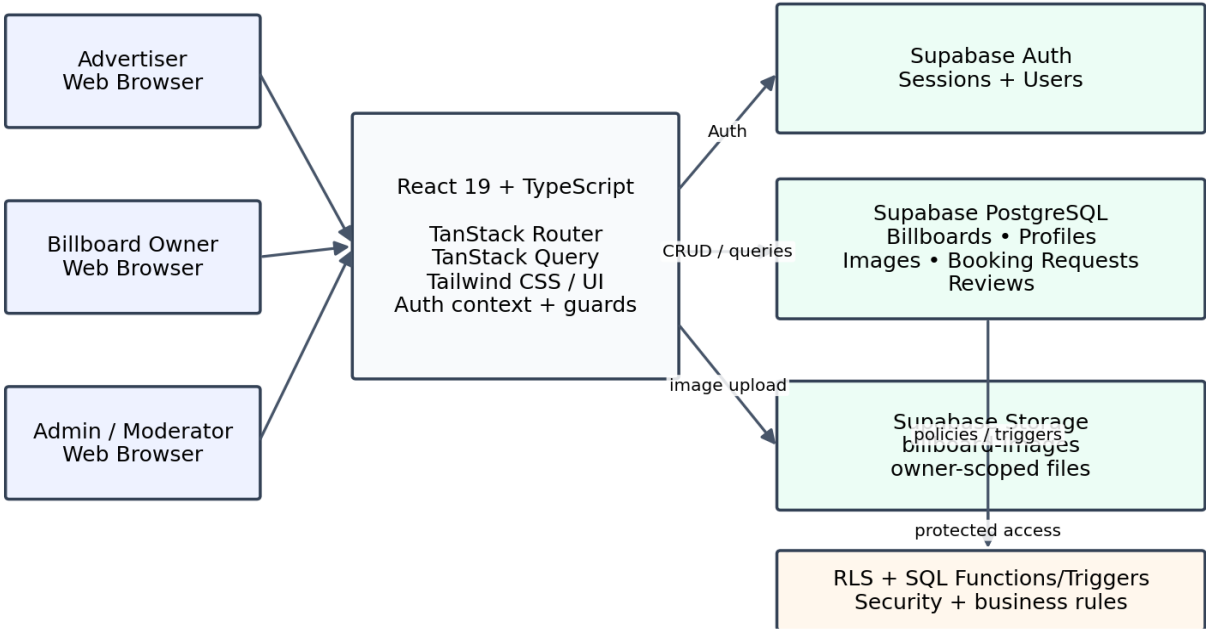
6. Users and Roles

Role	Responsibilities
Advertiser	Browse, search/filter, view details, save spaces locally, send requests, track requests, submit eligible reviews.
Billboard Owner	Create/edit/publish listings, upload images, manage requests and reply to reviews.
Admin / Moderator	Moderate reviews by hiding/restoring them with a required reason.

7. System Architecture

The application is a React/TypeScript web application backed by Supabase. TanStack Router handles routes, TanStack Query manages server state, and the authentication context coordinates sessions and roles.

Panorama — System Architecture

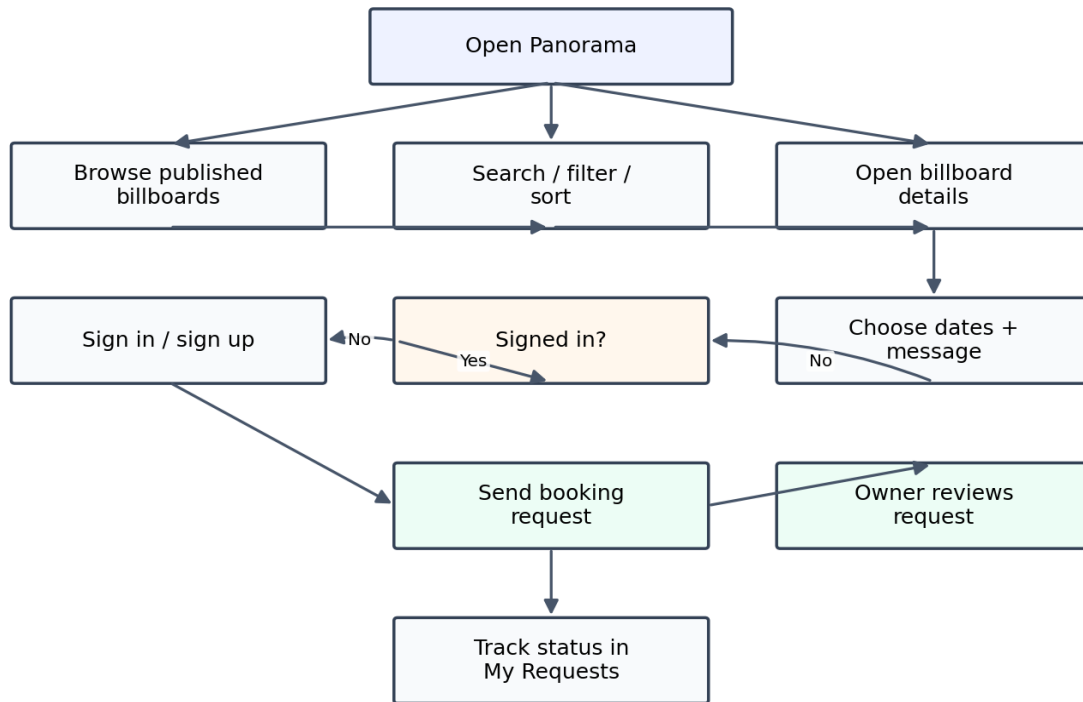


8. Technology Stack

Layer	Technology	Purpose
Frontend	React 19 + TypeScript	Web interface and application logic.
Build	Vite	Development and production bundling.
Routing	TanStack Router	Typed file-based routing.
Server state	TanStack Query	Queries, mutations and caching.
UI	Tailwind CSS + Radix UI	Responsive styling and accessible primitives.
Backend	Supabase	Backend platform.
Database	PostgreSQL	Relational application data.
Auth	Supabase Auth	Identity and sessions.
Storage	Supabase Storage	Billboard image files.
Maps	TomTom Web SDK	Location/map visualization.

9. Advertiser User Flow

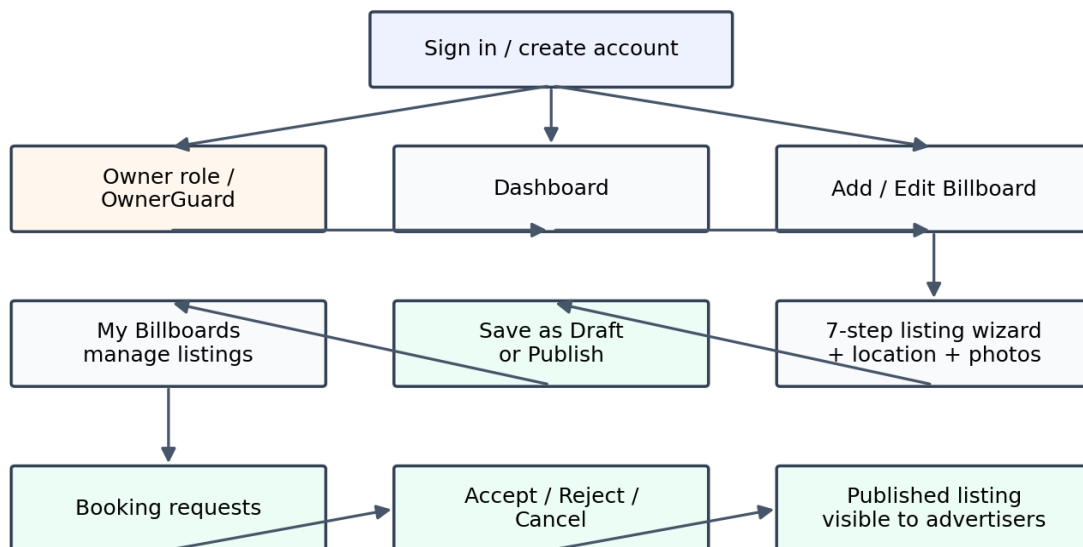
Panorama — Advertiser User Flow



Browsing is public. Protected actions trigger contextual authentication. Booking requests are stored with pending status and become visible to the owner.

10. Billboard Owner Flow

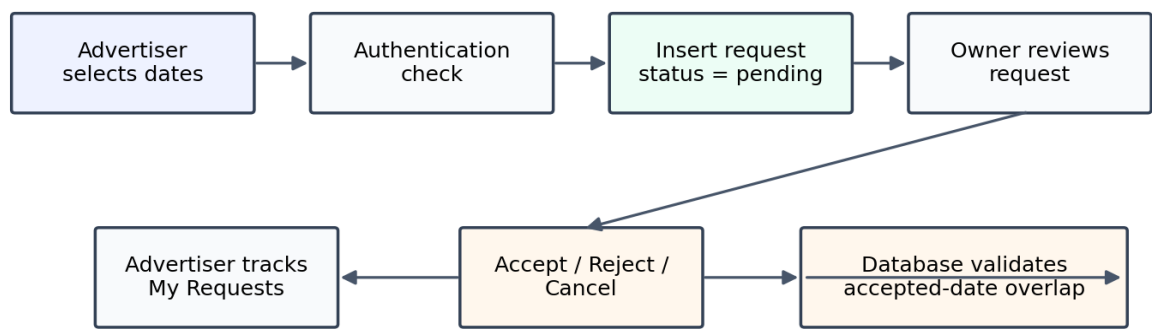
Panorama — Billboard Owner Flow



Owner-only routes are guarded in the UI and protected by database ownership rules.

11. Booking Request Flow

Panorama — Booking Request Flow



Client-side date validation is supplemented by database-level rules, including overlap prevention for accepted bookings.

12. Database Design

Panorama — Entity Relationship Diagram

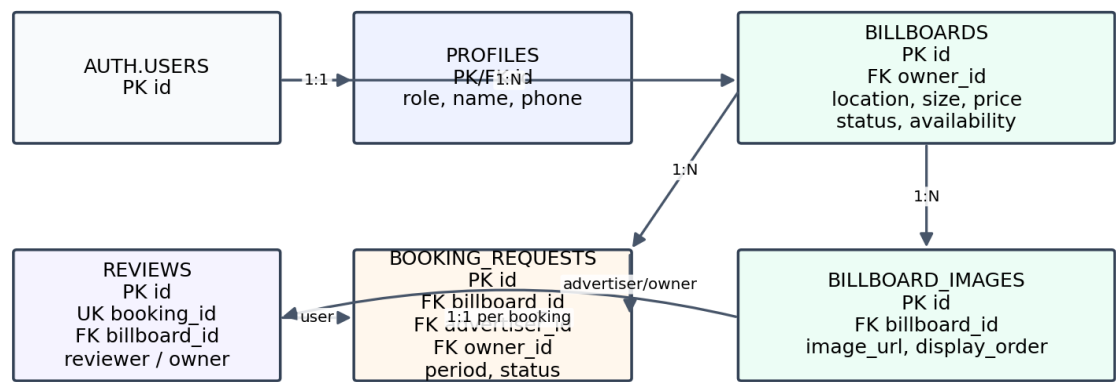


Table	Purpose	Key relationship
profiles	User application profile; role is advertiser or owner.	auth.users.id
billboards	Core billboard listing, location, dimensions, price, lighting, availability and public status.	profiles/auth.users
billboard_images	Image references and display order for each billboard.	billboards.id
booking_requests	Advertiser-to-owner booking requests with date range and status.	billboards.id + auth.users
reviews	Verified reviews tied to completed accepted bookings.	booking_requests.id + billboards.id + auth.users

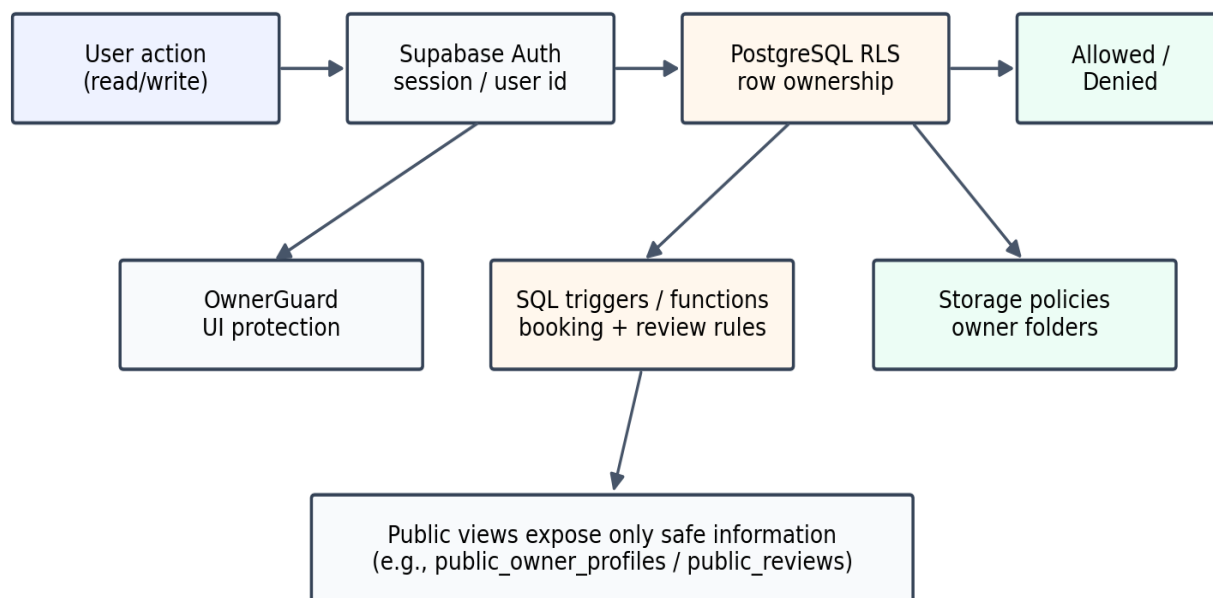
Table	Purpose	Key relationship
admins	Server-side moderator allowlist.	auth.users.id

13. Listing Lifecycle

- Draft — editable and private.
- Published — visible in marketplace and able to receive requests.
- Unpublished — hidden from discovery but retained for the owner.
- Deleted — listing and associated image records/files are removed.

14. Authentication and Security

Panorama — Authentication & Security Flow



- Supabase Auth establishes identity.
- RLS restricts database operations by user/ownership.
- OwnerGuard protects owner-only routes.
- Database functions/triggers enforce booking overlap and review eligibility.
- Public views expose limited owner/review information.
- Storage paths are scoped by user id.

15. Reviews and Trust System

Reviews are tied to verified transactions. Only the advertiser of an accepted booking whose period has ended can create a review. The database derives the billboard, owner and reviewer from the booking. Review content is immutable; an owner may reply once; moderators can hide a review with a reason.

16. Functional Modules

Authentication & Profiles: Supabase Auth session handling, profile creation, advertiser/owner roles, contextual sign-in gates and OwnerGuard route protection.

Marketplace: Public listing queries, search/filter/sort, featured billboards, popular cities, maps and billboard detail pages.

Owner Management: Dashboard, My Billboards, create/edit wizard, publishing controls, deletion, image synchronization and owner statistics.

Booking Requests: Date validation, request creation, advertiser request history, owner request queue and status updates.

Reviews & Moderation: Verified post-booking reviews, owner replies, review statistics, admin moderation and hidden-review handling.

Security: RLS policies, database triggers/functions, public-safe views, ownership checks and storage policies.

17. Testing and Verification

Area	Scenario	Result
Public browsing	Unauthenticated visitor opens home/browse pages and sees only published listings.	Pass / supported by published-status query and RLS.
Owner isolation	Owner fetches/manages only their own listings.	Pass / owner_id filters plus RLS.
Draft visibility	Draft/unpublished listings do not appear in public marketplace.	Pass / public query filters status=published.
Booking creation	Advertiser submits valid date range and optional message.	Pass / client validation + authenticated insert.
Booking overlap	Two accepted requests cannot overlap for the same billboard.	Database trigger/function prevents overlap.
Image ownership	Owner uploads images into their own storage folder.	Storage path uses user id and policies restrict access.
Review eligibility	Only advertiser of an accepted booking after its end date can review.	Database trigger enforces eligibility.
Review moderation	Admin can hide/restore reviews; hiding requires a reason.	Database policy/function enforces rule.

18. Advantages

- Centralized billboard discovery.
- Separate advertiser/owner workflows.
- Real database-backed inventory.
- Server-side authorization and business rules.
- Image and map support.
- Verified review model.
- Extensible architecture.

19. Current Limitations

- No online payment processing in supplied implementation.
- Favourites use browser localStorage rather than a server-side table.
- Availability is listing-date plus booking-request based.
- Notifications, invoicing, dispute handling and owner verification are future extensions.

20. Future Enhancements

- Online payments/escrow.
- Email/WhatsApp notifications.
- Geospatial radius search.

- Owner verification.
- Campaign analytics.
- Multi-billboard campaigns.
- Cross-device saved lists.
- Recommendation engine.
- Admin analytics.

21. Conclusion

Panorama demonstrates a practical digital marketplace model for outdoor advertising. Its implementation combines a responsive React frontend with Supabase authentication, PostgreSQL, storage and database-level security. Ownership, booking overlap and review eligibility are reinforced at the database layer, creating a strong base for further production development.

22. Selected Project Structure

Path	Role
src/routes/	Application pages and route workflows.
src/components/marketplace/	Marketplace UI and booking components.
src/components/owner/	Owner wizard, location picker and guard.
src/components/reviews/	Ratings, review and moderation UI.
src/lib/auth.tsx	Authentication/session context.
src/lib/queries.ts	Marketplace and booking data access.
src/lib/owner-queries.ts	Owner listing/image/dashboard data access.
src/integrations/supabase/client.ts	Supabase client.
supabase/migrations/schema.sql	Core schema.
supabase/migrations/security*.sql	RLS/security/business rules.
supabase/migrations/reviews.sql	Review schema/rules/views.

23. References / Implementation Sources

This documentation was generated from the supplied Panorama project archive, including its React/TypeScript source files, package manifest, Supabase migration SQL and README. Diagrams represent the architecture and workflows observable in those files.